Data Cleaning and Preparation

Slides for *MATH 251 – Database Queries*

Ali Tavasoli

Based on the slides and book by Antonio Badia
edited and revised by Stefanie Scherzinger & Ali Tavasoli

JAMES MADISON UNIVERSITY

Outline

1. Exploratory Data Analysis (EDA)
   - Univariate Analysis
   - Multivariate Analysis

Exploratory Data Analysis (EDA)

- The goal of **EDA** is to **understand your dataset before analysis**:
  - Identify the **data type**:
    structured, semi-structured, or unstructured
  - If structured, find the **schema**:
    Which **attributes (columns)** describe the data?
  - For each attribute, understand its **data type**:
    - Categorical (names, labels)
    - Ordinal (ordered categories)
    - Numerical (numbers)
  - Explore the **values**:
    - Typical value (average)
    - Range (min $\rightarrow$ max)
    - General pattern of the data

EDA (Cont.)

- For **structured data**, EDA has **two main steps**:
  - **Step 1: Univariate Analysis**
    Study **one attribute at a time**
  - First, identify the **domain type**:
    - Categorical → strings (e.g., names)
    - Numerical → numbers
    - Temporal → dates/times
    - Ordinal → strings or numbers (depends)
  - **Important:**
    Data types can be **misleading** → always **verify with EDA**
  - **Step 2: Multivariate Analysis**
    Study **relationships between attributes**

EDA: Univariate Analysis

- Once we know the **domain type**, we explore **its values**:
  - **Categorical data**:
    Use histograms / bar charts
  - **Numerical data**:
    Focus on:
    - Center → mean, median, mode
    - Spread → std, quartiles
  - **Optional:**
    Fit a distribution (e.g., normal) to summarize data
- **Also check for problems:**
  - Missing values
  - Outliers
  - Errors (wrong type, wrong values)
- These checks help with **data cleaning**

EDA: Univariate Analysis — Numerical Attributes

- We analyze **numerical data** using:
  - **Range**: minimum, maximum, range
  - **Center (central tendency)**: mean, median, mode
  - **Spread (dispersion)**: standard deviation, variance
  - **Shape (optional)**: skewness, kurtosis
- **Example (SQL summary): Attr is the attribute**

```
SELECT COUNT(Attr) AS number_values,
       COUNT(DISTINCT Attr) AS cardinality,
       MIN(Attr) AS minimum,
       MAX(Attr) AS maximum,
       MAX(Attr) - MIN(Attr) AS range,
       AVG(Attr) AS mean,
       STDDEV(Attr) AS standard_deviation
FROM Data;
```

Histograms: Frequency of Different Data Values

What is a Histogram?

- A histogram shows how data is **distributed**
- It groups values into **bins (ranges)**
- **Frequency**: each bar shows how many values fall in that range

Example: Exam Scores

Range	# Students
0–50	2
50–60	5
60–70	8
70–80	12
80–90	7
90–100	3

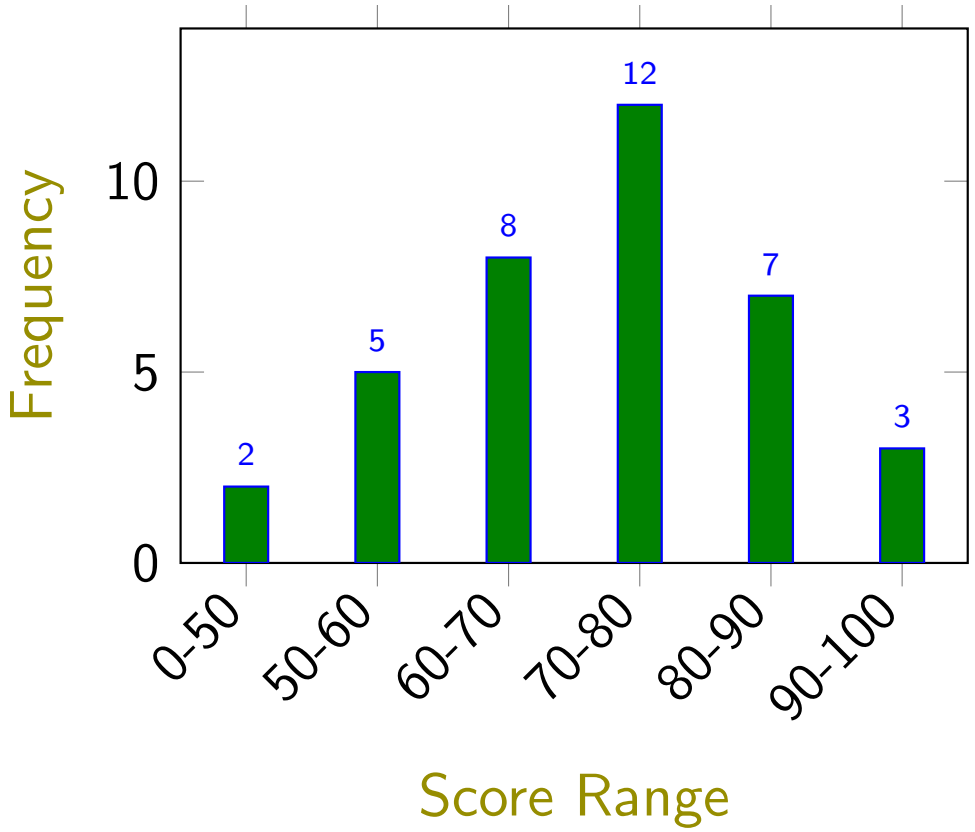
How to Read It

- **Taller bar** = more data in that range
- The **shape** shows the pattern:
  - **Symmetric** → balanced data
  - **Right-skewed** → long tail on the right
  - **Left-skewed** → long tail on the left

Key Idea: Histogram = "Where are most of the data values?"

Histogram: Visual Example

Exam Scores Distribution



How to Read It

- Each **bar** = count of data in a range
- **Taller bar** ⇒ more data
- Data grouped into **bins**

Pattern (Shape)

- **Symmetric** → balanced
- **Right-skewed** → tail on right
- **Left-skewed** → tail on left

Key Idea: Histogram = **"Where is most of the data?"**

Median: Middle/center of Ordered Data

Median

Odd Number of Values

2 4 **7** 9 12

Median = 7

Even Number of Values

2 4 **6** **8** 9 12

Median = (6 + 8)/2 = 7

How to Find It

- The **median** is the middle value
- First put the data in **ascending order**
- Then check how many values there are

Cases

- **Odd number** of values → one middle value
- **Even number** of values → average of the two middle values

Challenge

- For large datasets, sorting can take time
- In SQL, finding the median is harder than finding the mean

EDA: Numerical Attributes — Mean (Histogram)

Mean from a histogram

```
WITH Histogram(Value, Frequency) AS (
    SELECT Attr, COUNT(*)
    FROM Data
    GROUP BY Attr
)
SELECT (1.0 * SUM(Value * Frequency)) / SUM(Frequency) AS mean
FROM Histogram;
```

- We create a **histogram table** with two columns:
  - **Value**: each distinct value of the attribute
  - **Frequency**: number of times that value appears
- A **weighted average**: multiply each value by its frequency
- Divide by total number of values → **mean**

EDA: Numerical Attributes — Mean (Probability)

Mean from probabilities

```
WITH NHistogram(Value, Prob) AS (
    SELECT Attr, SUM(1.0 / total)
    FROM Data,
         (SELECT COUNT(*) AS total FROM Data) AS Temp
    GROUP BY Attr
)
SELECT SUM(Value * Prob) AS mean
FROM NHistogram;
```

- We create a table with:
  - **Value**: each distinct value
  - **Prob**: probability of that value (frequency ÷ total number of rows)
- This converts counts into **probabilities**
- Compute mean as: sum of (value × probability)

EDA: Mean via Probabilities (Step-by-Step)

Example dataset

$$\text{Data} = [2, 2, 3, 3, 3], \quad \text{total} = 5$$

Step 1: Attach total to each row (cross join)

```
Attr | total
------------
  2  |   5
  2  |   5
  3  |   5
  3  |   5
  3  |   5
```

Step 2: Compute probabilities

Each row contributes $\frac{1}{5}$, so grouping gives:

$$\mathbb{P}(2) = \frac{2}{5}, \quad \mathbb{P}(3) = \frac{3}{5}$$

Example

Trying this with dbfiddle:
https://www.db-fiddle.com/f/pqwqGBYGDGiGNhczcJh6R5/0

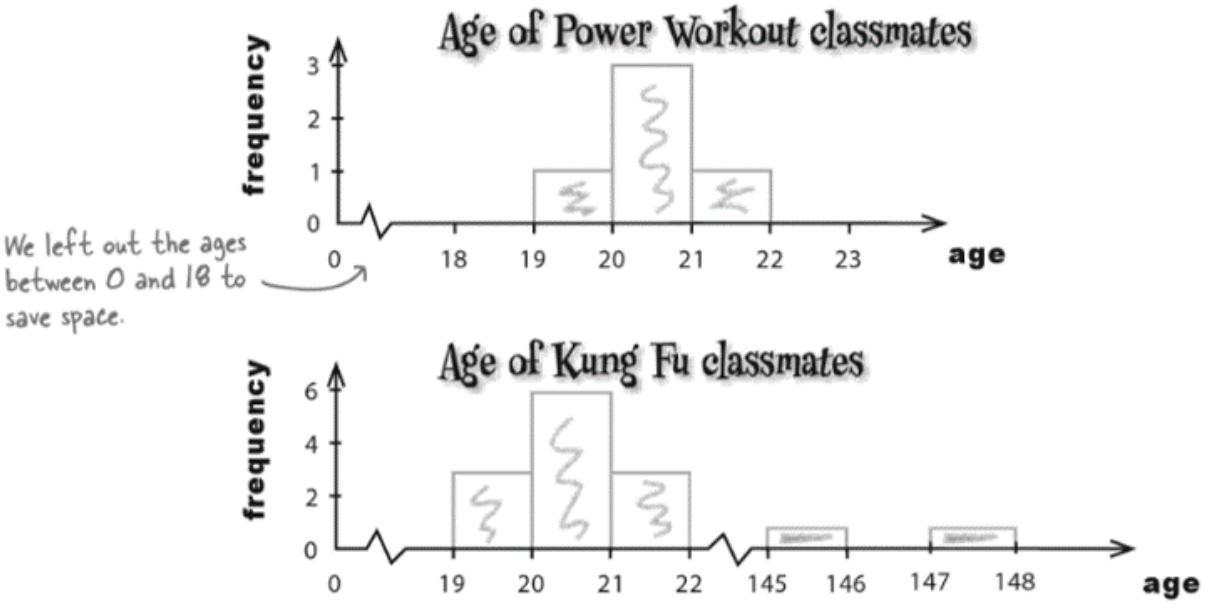
At the Health Club

Power Workout Classmate Ages

Age	19	20	21
Frequency	1	3	1

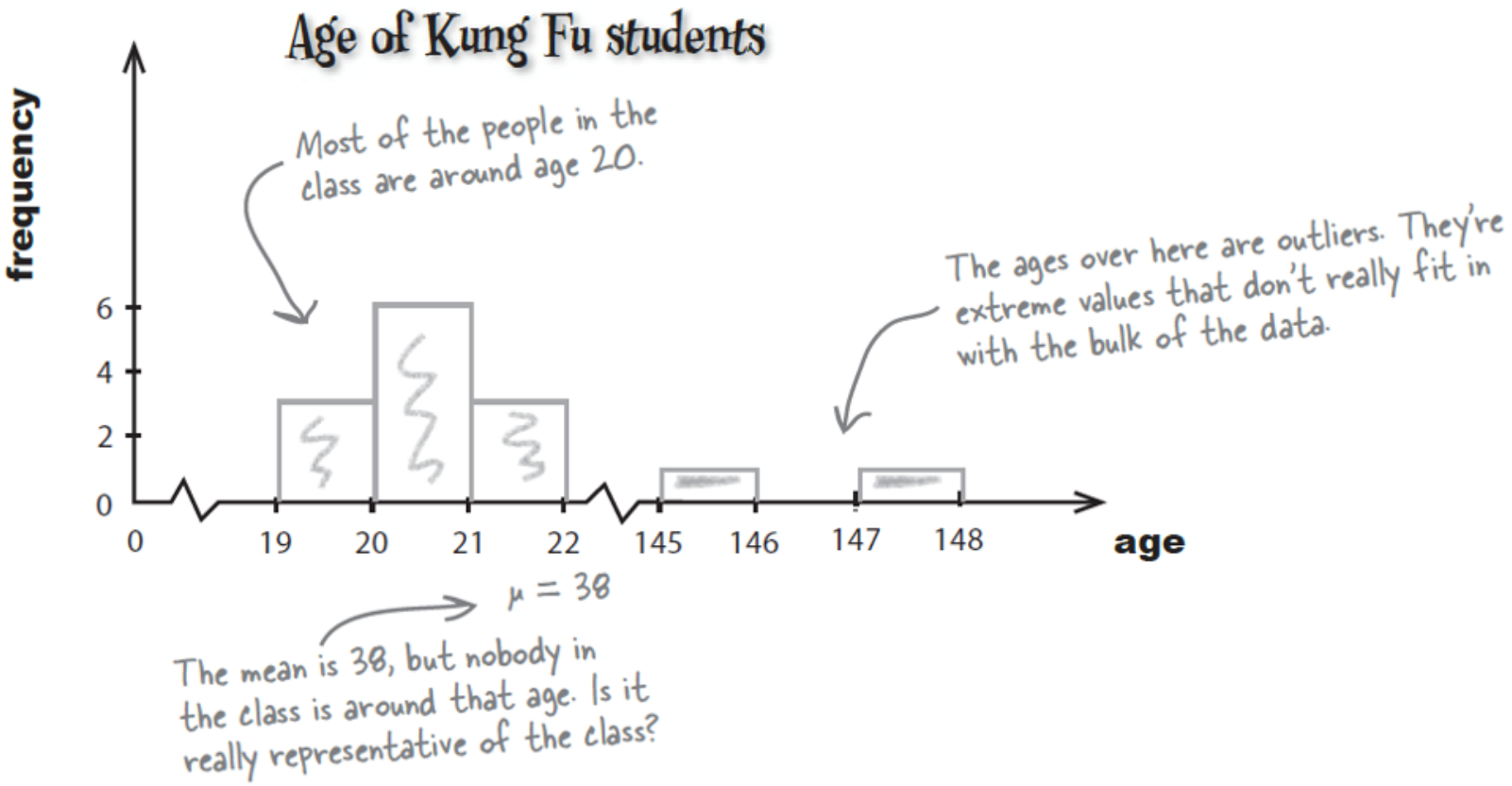
Kung Fu Classmate Ages

Age	19	20	21	145	147
Frequency	3	6	3	1	1



Source: Head First Statistics by Dawn Griffiths, O'Reilly.

At the Health Club

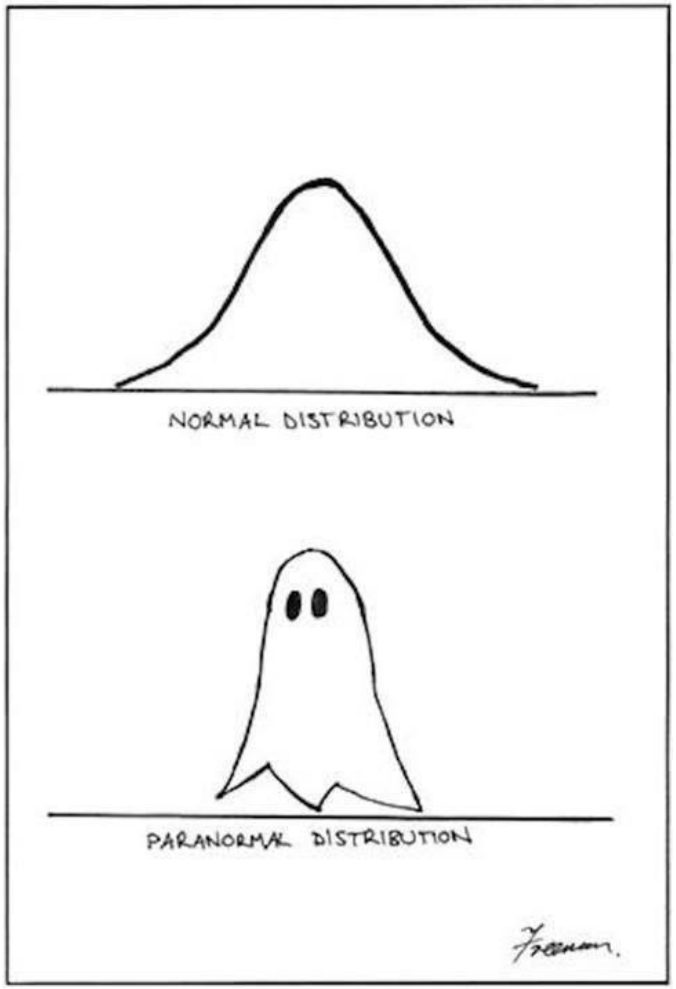


Source: Head First Statistics by Dawn Griffiths, O'Reilly.

Skewed Data

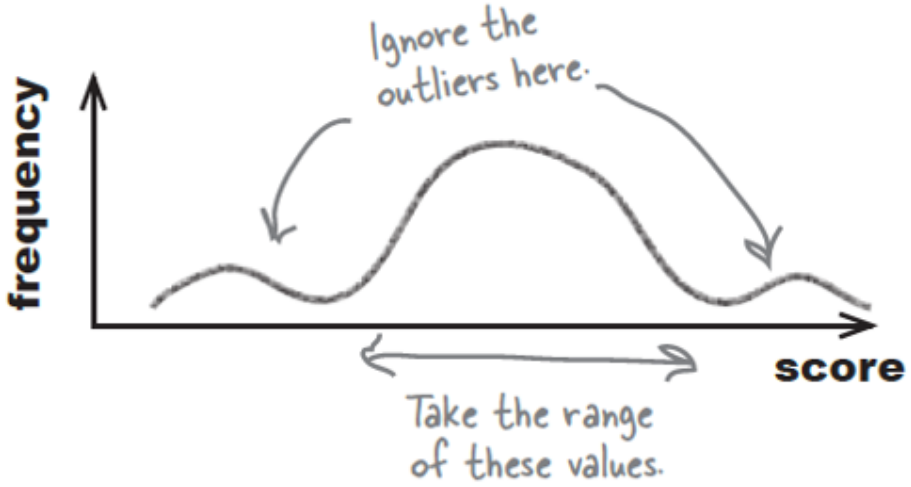
Source: Head First Statistics by Dawn Griffiths, O'Reilly.

Normal Distributions



https://www.rebresearch.com/blog/ab-normal-statistics-and-joke/.

Trimming Outliers



Source: Head First Statistics by Dawn Griffiths, O'Reilly.

EDA: Numerical Attributes — Trimmed Mean

Trimmed Mean

```
SELECT AVG(A)
FROM Data,
     (SELECT MAX(A) AS Amax FROM Data) AS T1,
     (SELECT MIN(A) AS Amin FROM Data) AS T2
WHERE A < Amax AND A > Amin;
```

Logic:

- Find the **largest** value and the **smallest** value
- Remove those **extreme values**
- Compute the average of the **remaining values**

Why do this?

It helps reduce the effect of **outliers**.

Try it in dbfiddle:

https://www.db-fiddle.com/f/442TTemmHKLqBK8u6mssHD/0

EDA: Univariate Analysis — Numerical Attributes — Mean

Geometric mean (gm) is useful when values change by **multiplication**, such as growth rates or interest rates.

- **Example:** interest rates over 5 years: 1%, 1.5%, 2%, 2.5%, 3%
- Convert them to **growth factors**: 1.01, 1.015, 1.02, 1.025, 1.03
- The **geometric mean** is $gm = (1.01 \times 1.015 \times 1.02 \times 1.025 \times 1.03)^{\frac{1}{5}}$
- If you invest **\$1000**, then after 5 years:

$$\$1000 \times 1.01 \times 1.015 \times 1.02 \times 1.025 \times 1.03 = \$1000 \times gm^5$$

- So, the geometric mean gives one **average multiplicative rate** for all 5 years.

Geometric Mean in SQL — Idea

Goal: Compute **geometric mean**

Challenge: SQL does **SUM**, not product

Key Trick: Logs

$$\ln(a \times b \times c) = \ln a + \ln b + \ln c$$

Step 1: Sum of logs

```
SELECT SUM(LN(Attr))
FROM Data;
```

Step 2: Take average

```
SELECT AVG(LN(Attr))
FROM Data;
```

Geometric Mean in SQL — Final Step

Step 3: Convert back using EXP

```
SELECT EXP(AVG(LN(Attr)))
FROM Data;
```

Final Idea:

$$gm = \exp\left(\text{avg}(\ln x)\right)$$

Interpretation:

- Log turns **product** $\rightarrow$ **sum**
- Average gives the **per-value rate**
- EXP returns to original scale

Example

Trying this with dbfiddle:
`https://www.db-fiddle.com/f/ifBmbzfDTqQptgaSUzSPVB/0`

EDA: Univariate Analysis — Numerical Attributes — Mean

Harmonic Mean is used for **rates** (like speed)
Example:

- Car goes at **60 mph** (first half)
- Returns at **30 mph** (second half)

Important: The car spends **more time** at 30 mph
Average speed is NOT 45 mph!
Correct average (Harmonic Mean): 40 mph
Key idea:

- Use harmonic mean when averaging **rates**
- It accounts for time differences

EDA: Univariate Analysis — Numerical Attributes — Mean

Harmonic Mean uses **reciprocals**

- For positive numbers $x_1, x_2, \ldots, x_n$, $\mathrm{HM} = \frac{n}{\frac{1}{x_1}+\frac{1}{x_2}+\cdots+\frac{1}{x_n}}$.
- **Example:** For $x_1 = 2$, $x_2 = 3$, $\mathrm{HM} = \frac{2}{\frac{1}{2}+\frac{1}{3}} = \frac{2}{\frac{5}{6}} = \frac{12}{5} = 2.4$.

Idea:

- Take reciprocal of each value: $\frac{1}{x}$
- Compute the **average**
- Take reciprocal again

SQL Implementation:

```
SELECT COUNT(Attr) / SUM(1.0/Attr)
FROM Data;
```

Key use: Averaging **rates** (speed, price per unit, etc.)

EDA: Univariate Analysis -Numerical Attributes -Mode

Mode = most frequent value

Step 1: Build Histogram (value, frequency)

```
WITH Histogram AS (
  SELECT Attr AS val, COUNT(*) AS freq
  FROM Data
  GROUP BY Attr
)
```

Step 2: Find highest frequency

```
SELECT val
FROM Histogram,
     (SELECT MAX(freq) AS top FROM Histogram) AS T
WHERE freq = top;
```

Output: value(s) with max frequency

EDA: Univariate Analysis -Numerical Attributes -Mode

Example: How Histogram works

Data (Attr):

`10, 20, 20, 30, 30, 30`

Step 1: Histogram (value, frequency)

val	freq
10	1
20	2
30	3

Step 2: Find highest frequency

- **MAX(freq)** $= 3$
- Value with freq $= 3 \rightarrow 30$

Mode = 30

EDA: Univariate Analysis -Numerical Attributes -Mode

Simpler case: one mode

```
SELECT Attr, COUNT(*) AS freq
FROM Data
GROUP BY Attr
ORDER BY freq DESC
LIMIT 1;
```

Key idea:

- **GROUP BY** → count values
- **ORDER BY** → highest first
- **LIMIT 1** → pick top

Example

Trying this with dbfiddle:
https://www.db-fiddle.com/f/mXQC8CtV6zYYbjZARdfJZ8/0

EDA: Univariate Analysis -Numerical Attributes -Median

Median = middle value after sorting

```
SELECT AVG(Attr)
FROM (
  SELECT Attr
  FROM Data
  ORDER BY Attr
  LIMIT 2 - MOD((SELECT COUNT(*) FROM Data), 2)
  OFFSET GREATEST(CEIL((SELECT COUNT(*) FROM Data) / 2.0 - 1),0)
) AS T;
```

- **ORDER BY** → sort values
- **OFFSET** → jump to middle
- **LIMIT** → take 1 or 2 values
- **AVG** → return median

Cases:

- **Odd size** → 1 middle value (LIMIT = 1)
- **Even size** → 2 middle values → average

EDA: Univariate Analysis - Numerical Attributes - Median

```
SELECT AVG(Attr)
FROM (
  SELECT Attr
  FROM Data
  ORDER BY Attr
  LIMIT 2 - MOD((SELECT COUNT(*) FROM Data), 2)
  OFFSET GREATEST(CEIL((SELECT COUNT(*) FROM Data) / 2.0 - 1),0)
) AS T;
```

- **MOD(n,2)** → checks **odd/even** (`MOD` remainder after division)
  - odd → 1 → LIMIT = 1
  - even → 0 → LIMIT = 2
- **CEIL(n/2 - 1)** → finds **middle index** (`CEIL` rounds a number up to the nearest integer)
  - shifts index to correct starting point
- **GREATEST(...,0)** → avoids **negative offset**
  - A defensive guard when the table is empty.
- **OFFSET** → skips to middle (`OFFSET k` ignores the first k rows, then start returning results.)
- **LIMIT** → selects 1 or 2 values
- **AVG** → final median

EDA: Univariate Analysis -Numerical Attributes -Standard Deviation

Built-in function in most SQL systems:

```
SELECT STDDEV(Attr) FROM Data;
```

What it measures: how spread out the data is

Formula:

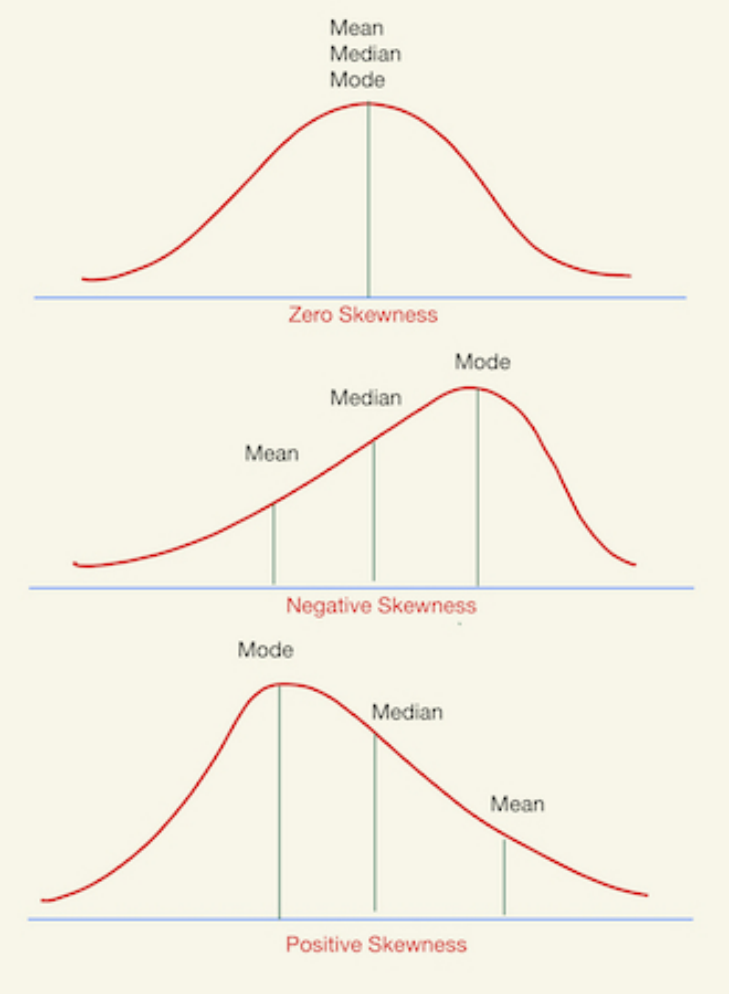
$$\sqrt{\frac{\sum_{i=1}^{n}(x_i - \mu)^2}{n-1}}$$

SQL version:

```
SELECT SQRT(SUM(POWER(Attr - mean, 2)) / (COUNT(*) - 1))
FROM Data, (SELECT AVG(Attr) AS mean FROM Data) AS T;
```

Idea: distance from mean $\rightarrow$ square $\rightarrow$ average $\rightarrow$ square root

Skew



https://study.com/academy/lesson/skewness-in-statistics-definition-formula-example.html

EDA: Univariate Analysis -Numerical Attributes -Skewness

Skewness = how asymmetric the data is

Interpretation:

- **Skewness** $> 0 \rightarrow$ tail to the right
- **Skewness** $< 0 \rightarrow$ tail to the left
- **Skewness** $\approx 0 \rightarrow$ symmetric

Formula for X: data values μ: mean of the data

$$\text{Skewness} = \frac{\mathbb{E}[(X-\mu)^3]}{(\mathbb{E}[(X-\mu)^2])^{3/2}}$$

```
SELECT SUM(POW(Value - mean, 3)) / COUNT(*) /
       POW(SUM(POW(Value - mean, 2)) / (COUNT(*) - 1), 3 / 2)
FROM Data, (SELECT AVG(Attr) AS mean FROM Data) AS T;
```

EDA: Univariate Analysis -Numerical Attributes -Kurtosis

Kurtosis = how heavy the tails / how peaked the data is

Interpretation:

- **High kurtosis** → heavy (long) tails extending far out, **more outliers**
- **Low kurtosis** → light (short) tails quickly shrinking , **fewer outliers**
- **Normal** → moderate tails (baseline)

Formula for X: data values μ: mean of the data

$$\text{Kurtosis} = \frac{\mathbb{E}[(X-\mu)^4]}{(\mathbb{E}[(X-\mu)^2])^2}$$

```
SELECT SUM(POW(Value - mean, 4)) / COUNT(*) /
       POW(SUM(POW(Value - mean, 2)) / (COUNT(*) - 1), 2)
FROM Data, (SELECT AVG(Attr) AS mean FROM Data) AS T;
```

EDA: Univariate Analysis – Categorical Attributes

Build a Histogram

SQL Pattern:

```
SELECT A, COUNT(*)
FROM R
GROUP BY A;
```

- **A** = categorical attribute
- **COUNT(*)** = frequency
- Groups rows with the same value of **A**

Goal: Count how many times each category appears

Example Table: `Students`

ID	Major	Year	GPA
1	CS	Soph	3.5
2	Math	Fresh	3.8
3	CS	Junior	3.2
4	Bio	Soph	3.6
5	CS	Senior	3.4
6	Math	Soph	3.7
7	Bio	Fresh	3.3

Histogram of `Major`

Major	Count
CS	3
Math	2
Bio	2

Interpretation: CS appears most frequently

EDA: Univariate Analysis – Entropy (Idea)

Entropy measures how much **uncertainty** or **disorder** exists in an attribute.

$$\mathbf{H(A)} = -\sum_{\mathbf{a} \in \mathbf{A}} \mathbf{P(a)} \log_2 \mathbf{P(a)}$$

- $\mathbf{P(a)}$ = probability of value *a*
- If one value dominates → **low entropy**
- If values are evenly spread → **high entropy**

Think:

- **Low entropy** = predictable
- **High entropy** = uncertain / mixed

EDA: Univariate Analysis – Entropy (SQL)

How to compute entropy in SQL:

Step 1: Compute probability of each value **Step 2:** Apply entropy formula

```
SELECT -SUM(Pa * LOG(2, Pa))
FROM (
    SELECT A, COUNT(*) * 1.0 / total AS Pa
    FROM Data
    CROSS JOIN (SELECT COUNT(*) AS total FROM Data) AS T
    GROUP BY A, total
) AS S;
```

What is happening?

- **GROUP BY A** → counts each value
- **1.0 / total** → builds probabilities
- **SUM(...)** → applies entropy formula

Try it: `https://www.db-fiddle.com/f/bKbjDc3ZewvgXFy4czvRmx/0`

EDA: Multivariate Analysis

Idea: Understand relationships between attributes

Depends on attribute types:

- **Categorical – Categorical**
  - Contingency tables
  - Chi-square test
- **Categorical – Numerical**
  - ANOVA (analysis of variance) checks whether different groups have significantly different averages.
  - Logistic regression
- **Numerical – Numerical**
  - Correlation, covariance
  - Linear regression
- **Ordinal – Ordinal**
  - Spearman's rank
  - Kendall's rank

EDA: Multivariate Analysis

Two ways to think about relationships

- **1. Cause & Effect (Dependence)**
  - One variable **influences** the other
  - **Independent (Predictor)** $\rightarrow$ input
  - **Dependent (Outcome)** $\rightarrow$ result
- **2. No Direct Dependence**
  - Variables may still be **related**
  - A third variable Z can affect both
  - This creates a **hidden connection**

Key Idea: Correlation does NOT imply causation

EDA: Multivariate Analysis - Joint Probability

Goal: Measure relationship between two attributes A and B
Key probabilities (SQL idea):

$$P(A) = \frac{\#A}{N}, \quad P(B) = \frac{\#B}{N}, \quad P(A, B) = \frac{\#(A, B)}{N}$$

SQL (Joint Probability):

```
SELECT A, B, SUM(1.0/total) AS JointProb
FROM Data, (SELECT COUNT(*) AS total FROM Data) AS T
GROUP BY A, B;
```

Idea: Create a table with $P(A)$, $P(B)$, $P(A, B)$
Independence Test: $P(A, B) = P(A)\,P(B)$
If true $\Rightarrow$ A and B are **independent**

EDA: Joint Probability - Example

Example: $A = \text{Gender}, \quad B = \text{Pass Exam}$

Data: Total– $N = 100$

Gender	Pass	Count
Male	Yes	30
Male	No	10
Female	Yes	20
Female	No	40

Probabilities: $P(\text{Male}) = \frac{40}{100}, \quad P(\text{Yes}) = \frac{50}{100}, \quad P(\text{Male, Yes}) = \frac{30}{100}$

Independence Check: $P(\text{Male, Yes}) \stackrel{?}{=} P(\text{Male}) \times P(\text{Yes})$

$$0.30 \neq 0.4 \times 0.5 = 0.20$$

Conclusion: Not independent $\Rightarrow$ there is a relationship

EDA: Multivariate Analysis - Independence Check

Goal: Check whether two attributes A and B are independent

Idea: If A and B are independent, then $P(A, B) = P(A)\,P(B)$ for all pairs (all combinations) of A and B values.

SQL idea:

- Compute $P(A)$, $P(B)$, $P(A, B)$
- Add up all differences (all combinations of A and B values):

$$|P(A, B) - P(A)P(B)|$$

If the final sum is 0: A **and** B **are independent**

Independence Check - Example (All Differences)

Example: $A = \text{Gender}, \quad B = \text{Pass}$

Probabilities:

$$P(\text{Male}) = 0.4,\ P(\text{Female}) = 0.6,\ P(\text{Yes}) = 0.5,\ P(\text{No}) = 0.5$$

Compute for each pair (A, B)**:** $|P(A, B) - P(A)P(B)|$

Pair	Calculation	Value
(Male, Yes)	$\lvert 0.30 - (0.4 \times 0.5)\rvert$	0.10
(Male, No)	$\lvert 0.10 - (0.4 \times 0.5)\rvert$	0.10
(Female, Yes)	$\lvert 0.20 - (0.6 \times 0.5)\rvert$	0.10
(Female, No)	$\lvert 0.40 - (0.6 \times 0.5)\rvert$	0.10

Sum of all differences: $0.10 + 0.10 + 0.10 + 0.10 = 0.40$

Conclusion: Sum $\neq 0 \Rightarrow$ **not independent**

EDA: Multivariate Analysis - Independence Check

```
WITH ProbA AS
   (SELECT A, SUM(1.0/total) AS PrA
    FROM Data, (SELECT COUNT(*) AS total FROM Data) AS T
    GROUP BY A),
  ProbB AS
   (SELECT B, SUM(1.0/total) AS PrB
    FROM Data, (SELECT COUNT(*) AS total FROM Data) AS T
    GROUP BY B),
  ProbAB AS
   (SELECT A, B, SUM(1.0/total) AS PrAB
    FROM Data, (SELECT COUNT(*) AS total FROM Data) AS T
    GROUP BY A, B)
SELECT SUM(ABS(PrAB - (PrA * PrB)))
FROM (SELECT ProbA.A, ProbB.B, PrA, PrB, PrAB
      FROM ProbA, ProbB, ProbAB
      WHERE ProbA.A = ProbAB.A AND ProbB.B = ProbAB.B) AS Probabilities;
```

EDA: Multivariate Analysis - PMI

Pointwise Mutual Information (PMI)

Idea: How surprising is it to see A and B together?

$$PMI(A, B) = \log_2 \frac{P(A, B)}{P(A)\,P(B)}$$

Interpretation:

- **PMI = 0** $\Rightarrow$ independent
- **PMI > 0** $\Rightarrow$ occur together **more than expected**
- **PMI < 0** $\Rightarrow$ occur together **less than expected**

SQL:

```
SELECT LOG(2, PrAB / (PrA * PrB))
FROM Probabilities;
```

EDA: Multivariate Analysis - PMI

```
WITH ProbA AS (
    SELECT A, SUM(1.0 / total) AS PrA
    FROM Data, (SELECT COUNT(*) AS total FROM Data) AS T
    GROUP BY A),
ProbB AS (
    SELECT B, SUM(1.0 / total) AS PrB
    FROM Data, (SELECT COUNT(*) AS total FROM Data) AS T
    GROUP BY B),
ProbAB AS (
    SELECT A, B, SUM(1.0 / total) AS PrAB
    FROM Data, (SELECT COUNT(*) AS total FROM Data) AS T
    GROUP BY A, B)
SELECT A, B,
       LOG(2, PrAB / (PrA * PrB)) AS PMI
FROM (
    SELECT ProbA.A, ProbB.B, PrA, PrB, PrAB
    FROM ProbA, ProbB, ProbAB
    WHERE ProbA.A = ProbAB.A
      AND ProbB.B = ProbAB.B) AS Probabilities;
```

EDA: Multivariate Analysis - Mutual Information

Mutual Information (MI)

Idea: Average of PMI over **all pairs** (A, B)

$$I(A, B) = \mathbb{E}_{P(A,B)}[PMI(A, B)]$$

Meaning:

- Measures **overall dependence**
- Always ≥ 0
- **0** $\Rightarrow$ independent

Intuition: PMI = one pair MI = **average over all pairs**

EDA: Numerical vs Numerical - Covariance

Goal: Measure how two numerical attributes move together

Covariance: $Cov(A, B) = \mathbb{E}\big[(A - \mathbb{E}(A))(B - \mathbb{E}(B))\big]$

Equivalent formula: $Cov(A, B) = \mathbb{E}(AB) - \mathbb{E}(A)\mathbb{E}(B)$

Interpretation:

- $> \mathbf{0} \Rightarrow$ move in **same direction**
- $< \mathbf{0} \Rightarrow$ move in **opposite direction**
- $\approx 0 \Rightarrow$ weak/no linear relation

SQL:

- Built-in:

```
SELECT covar_samp(A,B) FROM Data;
```

- Manual:

```
SELECT AVG(A*B) - (AVG(A)*AVG(B))
FROM Data;
```

EDA: Numerical vs Numerical - Correlation

Pearson Correlation is a normalized covariance (scale-free)

$$\rho = \frac{Cov(A, B)}{std(A)\, std(B)}, \quad \textbf{range} : -1 \leq \rho \leq +1$$

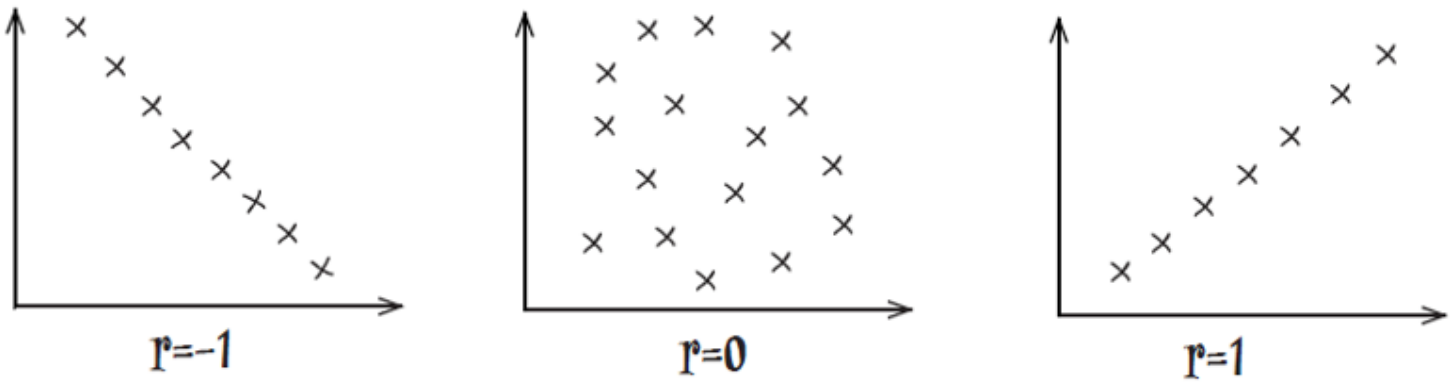
Interpretation:

- $\rho > 0 \Rightarrow$ move **together**
- $\rho < 0 \Rightarrow$ move **opposite**
- $|\rho|$ **large** $\Rightarrow$ strong relation
- $\rho \approx 0 \Rightarrow$ weak linear relation

Important:

- **If independent** $\Rightarrow \rho = 0$
- **If** $\rho = 0 \Rightarrow$ **no linear relationship**
- But there may still be a nonlinear relationship

Linear Relationships



From: "Heads First Data Statistics" by Dawn Griffiths, O'Reilly.

EDA: Numerical vs Numerical - Correlation (SQL)

Compute Correlation in SQL

Built-in (PostgreSQL):

```
SELECT CORR(A,B) FROM Data;
```

Manual (from definition):

```
SELECT (AVG(A*B) - AVG(A)*AVG(B))
       / (STD(A) * STD(B))
FROM Data;
```

Using covariance:

```
SELECT COVAR(A,B) / (STD(A) * STD(B))
FROM Data;
```

EDA: Multivariate Analysis - Ordinal vs Ordinal

- For **ordinal attributes**, we use **rank correlation**
- Idea: Compare how the **rankings** relate (not the raw values!)
- Suppose table `Data` has:
  - **A, B** (original attributes)
  - **Arank, Brank** (their ranks)
  - Ranks are: **1, 2, ..., n**
- Common measures:
  - **Spearman's** $\rho \rightarrow$ compares rank differences
  - **Kendall's** $\tau \rightarrow$ compares pairwise order
- **Takeaway:** Rank correlation = **relationship between orderings**

Example

Trying this with dbfiddle:
`https://www.db-fiddle.com/f/ma7RFSiU9GY5CiKyQ4tPuH/0`

Kendall's Rank Correlation (τ)

- **Goal:** Measure how similar two rankings are
- **Idea:** Look at **pairs of observations**
  - Compare every pair (i, j)
  - Check if the order in A and B agrees
- Two types of pairs:
  - **Concordant**: same order in A and B
  - **Discordant**: different order
- **Formula:**

$$\tau = \frac{\#\text{Concordant} - \#\text{Discordant}}{\text{Total number of pairs}}$$

- **Range:**
  - $\tau = 1 \rightarrow$ **perfect agreement**
  - $\tau = 0 \rightarrow$ **no relationship**
  - $\tau = -1 \rightarrow$ **opposite rankings**
- **Takeaway:** Counts how often rankings **agree vs disagree**

Kendall's τ - Concordant vs Discordant

- **Compare a pair of items**

Concordant (Same Order)

Item	A	B
I_1	1	2
I_2	2	3

In A: $1 < 2$

In B: $2 < 3$

Same order

Discordant (Flipped Order)

Item	A	B
I_1	1	2
I_2	2	1

In A: $1 < 2$

In B: $2 > 1$

Order flipped

- **Takeaway: Same order = Concordant — Flipped = Discordant**

Kendall's τ - Example

- **Data:**

Item	A rank	B rank
I_1	1	1
I_2	2	3
I_3	3	2

- **All pairs:** (I_1, I_2), (I_1, I_3), (I_2, I_3)
  - (I_1, I_2): Concordant
  - (I_1, I_3): Concordant
  - (I_2, I_3): Discordant
- **Compute:**

$$\tau = \frac{2-1}{3} = \frac{1}{3}$$

- **Interpretation:** More agreement $\rightarrow$ **positive correlation**

EDA: Multivariate Analysis - Ordinal/Ordinal

- **Kendall's τ:** $\tau = \frac{2\left(\#\text{concordant} - \#\text{discordant}\right)}{n(n-1)}$
- **SQL implementation:**

```
SELECT
2.0 * (
  SUM(CASE
        WHEN (D1.Arank < D2.Arank AND D1.Brank < D2.Brank)
          OR (D1.Arank > D2.Arank AND D1.Brank > D2.Brank)
        THEN 1 ELSE 0 END)
  -
  SUM(CASE
        WHEN (D1.Arank < D2.Arank AND D1.Brank > D2.Brank)
          OR (D1.Arank > D2.Arank AND D1.Brank < D2.Brank)
        THEN 1 ELSE 0 END)
)
/
(COUNT(*) * (COUNT(*) - 1))
FROM Data D1, Data D2
WHERE D1.Arank <> D2.Arank;
```

- **Takeaway:** SQL is just counting agreements vs disagreements

EDA: Multivariate Analysis - Ordinal/Ordinal

- **Spearman's ρ:** how far ranks are apart (**distance**)

$$\rho = 1 - \frac{6 \sum_i d_i^2}{n^3 - n}$$

- **Where:**
  - $d_i = \text{Arank}_i - \text{Brank}_i$
  - d_i = difference between the two rankings
- **Idea:**
  - Small differences $\rightarrow$ similar rankings
  - Large differences $\rightarrow$ different rankings
- **SQL implementation:**

```
SELECT
  1 - (6 * SUM(POW(Arank - Brank, 2))) /
      (POW(COUNT(*), 3) - COUNT(*))
FROM Data;
```

EDA: Multivariate Analysis - Categorical → Numerical

- **Question:** Does **fertilizer type** affect **plant growth**?
- **Data:** Table `Growth(fertilizer, height)`
- **Idea (ANOVA: Analysis of Variance):** Compare **means across groups**
- **Step 1: Group means**

```
CREATE TABLE group_means AS
SELECT fertilizer,
       AVG(height) AS group_mean
FROM Growth
GROUP BY fertilizer;
```

- **Step 2: Overall mean**

```
SELECT AVG(group_mean) AS overall_mean
FROM group_means;
```

- **Takeaway:** Check if group means are **different from overall**

EDA: Multivariate Analysis - Categorical → Numerical

- **Step 3: Between-groups variation**— Now compare each group mean to the overall mean
- **Formula (idea):** $\frac{\sum(\text{group mean} - \text{overall mean})^2}{\text{number of groups} - 1}$
- **SQL:**

```
SELECT
  SUM(POW(group_mean - overall_mean, 2)) * MAX(size) /
  (MAX(num_groups) - 1) AS between_groups
FROM (SELECT COUNT(*) AS size FROM Growth) T1,
     (SELECT COUNT(DISTINCT fertilizer) AS num_groups
      FROM Growth) T2,
     (SELECT AVG(group_mean) AS overall_mean
      FROM group_means) T3,
     group_means;
```

- **Interpretation:** Large value → group means are **very different**

EDA: Multivariate Analysis - Categorical → Numerical

- **Step 4: Within-groups variation**
- **Recall (previous slide):**
  - We measured how far **group means** are from the overall mean
  - → this was **between-groups (signal)**
- **Now:** Measure variation **inside each group**
- **Idea:**
  - Compare each **height** to its **group mean**
  - Large variation → noise within groups
- **SQL:**

```
SELECT
  SUM(POW(height - group_mean, 2)) /
  (num_groups * (size - 1)) AS within_groups
FROM Growth, group_means
WHERE Growth.fertilizer = group_means.fertilizer;
```

- **Interpretation:** How spread out data is **inside each group** → **noise**

EDA: Multivariate Analysis - Categorical $\rightarrow$ Numerical

- **Step 5: F-ratio (ANOVA)**
- **Recall:**
  - **Between-groups** $\rightarrow$ differences across groups (**signal**)
  - **Within-groups** $\rightarrow$ variation inside groups (**noise**)
- **F-ratio:**

$$F = \frac{\text{Between-groups variability}}{\text{Within-groups variability}}$$

- **Interpretation:**
  - Large F $\rightarrow$ groups are **very different** (strong effect)
  - Small F $\rightarrow$ differences are **mostly noise**

- **Takeaway:** F **= signal / noise**

EDA: Categorical → Numerical (F-ratio - Setup)

To put it all together

- **Step 1: Precompute components**

```
WITH group_means AS (
  SELECT fertilizer, AVG(height) AS group_mean
  FROM Growth
  GROUP BY fertilizer
),
stats AS (
  SELECT
    COUNT(*) AS size,
    COUNT(DISTINCT fertilizer) AS num_groups
  FROM Growth
),
overall AS (
  SELECT AVG(group_mean) AS overall_mean
  FROM group_means
)
```

- **We now have:** group means, overall mean, sizes

EDA: Categorical → Numerical (F-ratio)

- **Step 2: Compute F-ratio**

```
SELECT
  between_groups / within_groups AS F_ratio
FROM
-- Between-groups (signal)
(
  SELECT
    SUM(POW(group_mean - overall_mean, 2)) * size /
    (num_groups - 1) AS between_groups
  FROM group_means, stats, overall
) AS b,
-- Within-groups (noise)
(
  SELECT
    SUM(POW(height - group_mean, 2)) /
    (num_groups * (size - 1)) AS within_groups
  FROM Growth
  JOIN group_means
  ON Growth.fertilizer = group_means.fertilizer,
       stats
) AS w;
```

EDA: Categorical → Numerical (F-ratio)

Idea

- Between = signal
- Within = noise
- F = signal / noise

Steps

- group means
- overall mean
- compute variances

```
-- Final SQL (compressed)
WITH group_means AS (...)
SELECT between_groups / within_groups
FROM ...
```

EDA: Multivariate Analysis – Categorical/Categorical

- When both attributes are **categorical**, we compare their **counts**.
- Organize counts in a **contingency table** (**cross-tabulation**).
- Goal: **Are the variables related?**
- Use **Chi-square statistic**: χ^2 **test of independence**
- Example: contingency table `CUSTOMERS(City, Status, Count)`

- `Status`: **Active** / **Stopped**
- Question: **Does city affect subscription?**

CUSTOMERS		
City	**Status**	**Count**
Gotham	Active	1462
Gotham	Stopped	794
Metropolis	Active	749
Metropolis	Stopped	385
Smallville	Active	527
Smallville	Stopped	139

EDA: Multivariate Analysis – Categorical/Categorical

Goal: Does City affect Status?

1. **Step 1: Margins**
   Compute totals: **by City**, **by Status**, and **overall**
2. **Step 2: Overall rates**
   Active rate and **Stopped rate** = count ÷ total
3. **Step 3: Expected counts**
   For each city: **Expected = (city total) × (overall rate)**
4. **Step 4: Deviation**: **Observed − Expected**
5. **Step 5: χ^2 contribution** $\dfrac{(\textbf{Observed} - \textbf{Expected})^2}{\textbf{Expected}}$

Step 1: Margins

Goal: Compute totals

- **By City**:
  - Gotham: 2256
  - Metropolis: 1134
  - Smallville: 666
- **By Status**:
  - Active: 2738
  - Stopped: 1318
- **Total**: **4056**

City	Status	Count
Gotham	Active	1462
Gotham	Stopped	794
Metropolis	Active	749
Metropolis	Stopped	385
Smallville	Active	527
Smallville	Stopped	139

Step 2: Overall Rates

- **Active rate**:

$$\frac{2738}{4056} \approx 0.675$$

- **Stopped rate**:

$$\frac{1318}{4056} \approx 0.325$$

- Interpretation: **About 67.5% active, 32.5% stopped overall**

Step 3: Expected Counts (Gotham)

- City total (Gotham): 2256
- **Expected Active**:

$$2256 \times 0.675 \approx 1523$$

- **Expected Stopped**:

$$2256 \times 0.325 \approx 733$$

- Compare with observed:
  - Active: 1462 vs 1523
  - Stopped: 794 vs 733

Step 4-5: Deviation and χ^2

- **Deviation**:

Observed - Expected

- Gotham (Active):

$$1462 - 1523 = -61$$

- χ^2 **contribution**:

$$\frac{(-61)^2}{1523} \approx 2.44$$

- Repeat for all cells and sum them:

$$\chi^2 = \sum \frac{(O-E)^2}{E}$$

Step 4-5: Deviation and χ^2 (Active Only)

- **Deviation**:
  Observed - Expected
- χ^2 **contribution**: $\frac{(O-E)^2}{E}$
- Add all contributions:
  $\chi^2 = \mathbf{4.7}$

Active Status			
City	**O**	**E**	$\frac{(O-E)^2}{E}$
Gotham	1462	1523	2.44
Metropolis	749	765	0.33
Smallville	527	450	1.93

(values rounded)

- **Interpretation:** Some difference $\Rightarrow$ possible relationship
  - Measures how far data is from **independence**
  - **Closer to 0** $\Rightarrow$ closer to independence
  - **Large values** $\Rightarrow$ stronger relationship
- Back to initial question: **Does City affect subscription?**
  - Gotham: lower Active than expected $\Rightarrow$ worse subscription
  - Metropolis: close to expected $\Rightarrow$ no clear effect
  - Smallville: higher Active than expected $\Rightarrow$ better subscription

EDA: Multivariate Analysis -Categorical/Categorical

In SQL, this can be written in a single query, using WITH and subqueries in the FROM clause to organize the margin sum calculations:

```
WITH
 (SELECT Cust.city, Cust.status, Cust.Count,
         perCity * (perStatus / total) AS expected
  FROM Cust,
       (SELECT city, SUM(Count) AS perCity
        FROM Cust
        GROUP BY city) AS Cities,
       (SELECT status, SUM(Count) AS perStatus
        FROM Cust
        GROUP BY status) AS Statuses,
       (SELECT SUM(Count) AS total FROM Cust)
  WHERE Cust.city = Cities.city
        AND Cust.status = Statuses.status)
SELECT SUM(POW(Count-expected,2)/expected);
```

EDA: Distribution Fitting

- **Goal:** test whether an attribute is well described by a **chosen distribution**
- **Step 1:** Build the empirical distribution

```
(value, probability)
```

- **Step 2:** Compute key parameters of the empirical distribution, e.g., **mean** and **standard deviation**
- **Step 3:** Add a fitted column

```
(value, probability, estimate)
```

- **Step 4:** Use the model formula to compute **estimate**
- **Step 5:** Compare **estimate** with **probability**
- **Small difference** $\Rightarrow$ **good fit**

Example: Fitting a Normal Distribution

- **Raw data (exam scores)**

  60, 70, 70, 80, 80, 80, 80, 90, 90, 100

- **Empirical distribution**

```
value | probability
-------------------
  60  |    0.10
  70  |    0.20
  80  |    0.40
  90  |    0.20
 100  |    0.10
```

- Probability = frequency / total

- **Fit a model** = choose a distribution shape and estimate its parameters from the data
- Example: normal distribution

$$\mu = 80, \qquad \sigma \approx 12$$

- These parameters define the fitted model:

$$X \sim \mathcal{N}(80, 12^2)$$

- For each value, compare estimates with observed probabilities

Example: Normal Fit in Practice

- **Observed vs fitted values**

```
value | probability | estimate
-------------------------------
  60  |   0.10      |   0.09
  70  |   0.20      |   0.21
  80  |   0.40      |   0.40
  90  |   0.20      |   0.21
 100  |   0.10      |   0.09
```

- **Interpretation**
- The estimate values are close to the empirical probability
- So the data is **reasonably well fit** by a Normal model

- **SQL idea**
- First compute the empirical distribution:

```
(value, probability)
```

- Then compute AVG(value) and STDDEV(value)
- Add a new column:

```
estimate
```

- Fill it using the Normal formula with the sample mean and standard deviation

EDA: Distribution Fitting (Cont.)

- **Example**
- A phone company records call lengths
- Raw data:

  `(callid, minutes)`

- We convert it to a distribution:

  `(minutes, probability)`

- Each probability = frequency / total

- **SQL (build distribution)**

```
CREATE TABLE DATA AS
SELECT
  minutes,
  COUNT(*) * 1.0 / total AS probability
FROM RawData,
     (SELECT COUNT(*) AS total
      FROM RawData) t
GROUP BY minutes, total;
```

- **Output:**

  `(minutes, probability)`

EDA: Distribution Fitting (Cont.)

- **Add model table**
- Create a table with:

  `(value, observed, estimate)`

- Fill:
  - observed ← data
  - estimate ← NULL
- **Compute parameters**
- Use: `AVG` and `STDDEV`

- **SQL**

```
CREATE TABLE NORMAL AS
SELECT
  minutes AS value,
  freq AS observed,
  NULL AS estimate
FROM DATA;

SELECT
  AVG(minutes) AS mu,
  STDDEV_SAMP(minutes) AS sigma
FROM RawData;
```

- **Next:** use μ, σ to fill `estimate`

EDA: Distribution Fitting (Cont.)

- **Fill model estimates**
- Use the Normal distribution formula with μ and σ
- Compute expected values for each `value`
- **Compare**
- Compare observed vs estimate
- **Small difference** $\Rightarrow$ **good fit**

- **SQL**

```
UPDATE NORMAL
SET estimate =
 (1.0 / (sigma * SQRT(2 * PI()))) *
 EXP(
  - POWER(value - mu, 2) /
    (2 * POWER(sigma, 2))
 );
```

- **Note:** uses the Normal PDF[a]

$$\mathcal{N}(\mu, \sigma^2)$$

[a] Probability Density Function